



ISLAMIC UNIVERSITY OF TECHNOLOGY (IUT)
ORGANISATION OF ISLAMIC COOPERATION (OIC)
DEPARTMENT OF ELECTRICAL AND ELECTRONIC
ENGINEERING

Project Title:

Morse Code Generator and Decoder with graphical user interface

Group No: **B2_Group3**

Members:

- Sayeed Hossain Sayor
200021236
- M.M. Sadman Shafi
200021240
- MD Wahiduzzaman
200021228
- Shafayet Sadik Sowad
200021248
- Mohammad Saed Jama
200021260

2. Project Problem Statement:

Morse code remains a vital communication method in various fields, requiring efficient encoding and decoding systems. This project aims to develop a Graphical User Interface (GUI) in MATLAB to handle Morse code generation and decoding.

The project consists of two parts:

1. **Morse Code Generator:** The GUI will allow users to input a text message (up to 10 characters) and convert it into Morse code, displaying the corresponding symbols and playing the encoded sound. The entire Morse message will be saved as an audio file (.wav format).
2. **Morse Code Decoder:** The GUI will enable users to upload a Morse code audio file, analyze it, and decode the message back into text for display.

This project provides an interactive tool for Morse code communication, bridging text and audio-based messaging efficiently.

3.Solution Abstract

Morse code encoding and decoding represent fundamental communication methods used in various applications. The proposed solution offers a comprehensive approach for both encoding text to Morse code and decoding Morse code from an audio signal. It provides a user-friendly interface with selectable modes for encoding and decoding, where users can input either text or Morse code for conversion. In encoding mode, the solution converts user-provided text into its Morse code representation using a predefined structure. Each character is translated into its corresponding Morse symbol (dot or dash), and the resulting sequence is played back as an audio signal. The audio signal is generated using sine waves to represent dots and dashes, with appropriate pauses for separating characters and words. The encoded message is saved as a .wav file, allowing for both auditory verification and preservation of the Morse code. In decoding mode, the solution handles the reverse process, where Morse code is inputted and translated back into the original text. The program converts Morse code into its text equivalent and plays the decoded audio for auditory confirmation. This mode also supports common punctuation symbols, further enhancing the system's versatility in handling various Morse code scenarios.

For decoding Morse code from an audio signal, the solution implements a robust method to process the input audio file. First, it detects the tone frequency of the signal by applying a bandpass filter, which isolates the relevant frequency components. The Hilbert transform is then used to compute the envelope of the signal, which is thresholded to identify the on/off states corresponding to dots and dashes. The system also considers the duration of each symbol, classifying them as either dots (dots) or dashes (dashes). Outliers in signal durations are filtered out to ensure accurate classification.

Silent intervals between characters and words are also detected to ensure proper segmentation of the Morse code. The decoding process dynamically adjusts to the detected tone frequency, allowing it to work with varying input signal characteristics. Finally, the identified dots, dashes, and spaces are converted into a readable Morse code message, with word separation indicated by slashes (/). This approach ensures that the system can efficiently decode Morse code from audio recordings, offering a reliable and adaptive solution for both encoding and decoding tasks in Morse code communication.

4.Detailed Methodology

Part-1

Morse Code Data Structure

The Morse code system is represented using a MATLAB structure `morse`, where each field corresponds to a character in the alphabet or a numeral. This structure allows for quick lookups for Morse code translations:

- The lowercase English alphabet (a-z) is mapped to its Morse code equivalent.
- Digits (0-9) are also encoded using predefined Morse code symbols (e.g., '1' as `.----`, '2' as `..---`, etc.).
- Common punctuation marks are encoded using a predefined set of Morse code symbols stored in `morse.scv`.

User Input

The program prompts the user to choose between two modes: Encoding (e) and Decoding (d). Based on the selected mode, the following steps are carried out:

Encoding Process

In the encoding mode:

- The user is prompted to input a string of text, which is then converted to lowercase using the `lower()` function to standardize the input.
- The string is processed character by character. For each character:
- If the character is a space, a slash (/) is appended to the Morse code sequence to represent the word separation.
- For valid alphanumeric characters, the corresponding Morse code is retrieved using the `getfield()` function to access the `morse` structure. If the character is a letter, the `getfield(morse, text(i))` retrieves the respective Morse code, and for digits, the field `['n', text(i)]` is accessed.
- Common punctuation symbols are handled by checking the presence of the symbol in `morse.sc` and retrieving the corresponding Morse code from `morse.scv`.

Audio Signal Generation

Once the entire text string is converted into Morse code:

- Audio signals for dots and dashes are generated using sine waves. Specifically:
 - A dot (.) is represented by a short sine wave (dot), and a dash (-) by a longer sine wave (dash).
 - Silent pauses (ssp for short pauses and lsp for longer pauses) are used to separate symbols, characters, and words.
- The entire Morse code sequence is mapped to a corresponding sound waveform by concatenating the generated sine waves.
- The sound is played back using the `audioplayer()` function, which takes the generated sound waveform and the sample rate (13 kHz in this case) as input parameters.

Additionally, the sound is saved as a .wav file using the `audiowrite()` function, ensuring that the encoded Morse code can be stored for future use or auditory verification.

Decoding Process

In decoding mode:

- The user inputs a Morse code sequence, which is processed character by character to generate the corresponding audio representation (similar to the encoding process).
- A loop iterates over the Morse code input. For each symbol:
 - Dots and dashes are converted into their respective sine wave sounds, and spaces are used to represent silent intervals.
- The audio corresponding to the input Morse code is played back, allowing the user to hear the code being processed.

Morse Code Translation

After generating the audio signal, the program attempts to decode the Morse code:

- The input Morse code is split into characters based on spaces and slashes, which represent character and word boundaries, respectively.
- For each Morse code sequence (representing a character), the program compares it against the predefined Morse codes in the `morse` structure.
 - If a match is found for a letter, number, or punctuation, the corresponding character is appended to the decoded message.
 - If a space (/) is encountered, a space is added to the decoded message to indicate word separation.

The decoded output is then displayed to the user.

Signal Processing Techniques

To generate Morse code sounds, the following signal processing techniques are used:

- **Sine Wave Generation:** The sine waves representing dots and dashes are generated using MATLAB's `sin()` function. The sine wave frequencies are determined by the length of the dot and dash signals. A shorter sine wave is used for dots, and a longer one for dashes.
- **Signal Concatenation:** The individual sine wave segments are concatenated to form the entire Morse code sequence.
- **Pause Management:** Silent pauses are added between symbols (dots and dashes), between characters (short pause), and between words (long pause). These pauses ensure that the generated sound follows the standard Morse code timing conventions, making it recognizable when played back.

Audio Playback and File Saving

The `audioplayer()` function is used to play the generated Morse code audio. It accepts the sound waveform (ms) and the sample rate (13 kHz in this case) as input. This function is ideal for handling audio playback directly from MATLAB. Furthermore, the sound is saved to a .wav file using the `audiowrite()` function, providing the user with a reusable audio file of the Morse code message.

Error Handling and Robustness

- **Space and Punctuation Handling:** The program handles spaces and punctuation marks efficiently, ensuring that each character or symbol is correctly translated into Morse code or vice versa.
- **Signal Duration Filtering:** While the code does not explicitly implement duration filtering in the context of Morse code decoding, it inherently handles various durations of dot and dash signals by managing pauses between symbols and characters.
- **Variable-Length Input:** The system can process strings of arbitrary length, and the pause duration is automatically adjusted based on the length of the input Morse code.

Part-2

For the second part of the project, we will first describe the relevant signal processing concepts and then provide a detailed methodology for their application in this project.

1. Power Spectral Density (PSD)

Power Spectral Density (PSD) refers to the distribution of power of a signal over different frequencies, which helps in identifying dominant frequencies and filtering out noise. It is typically calculated by applying the Fourier Transform to the signal, converting it into the frequency domain to analyze the magnitude of frequency components. In this project, PSD is used to detect the Morse code tone frequency by dividing the audio signal into smaller segments and calculating the power spectral density for each segment. The frequency with the highest power is identified as the Morse code tone, which is crucial for isolating the relevant signal from surrounding noise. The input signal is processed by applying a sliding window, multiplying each segment with a Hanning window, and performing the Fast Fourier Transform (FFT). The peak frequency is then extracted from the averaged PSD values, which corresponds to the tone frequency of the Morse code.

2. Fast Fourier Transform (FFT)

The Fast Fourier Transform (FFT) is an efficient algorithm for calculating the Discrete Fourier Transform (DFT) of a signal. It transforms a time-domain signal into its frequency components, allowing us to analyze the frequency content of the signal. In this project, FFT is used to transform the audio signal from the time domain to the frequency domain, facilitating the identification of the Morse code tone frequency. The signal is divided into overlapping windows, each window is multiplied by a Hanning window to reduce spectral leakage, and the FFT is performed to extract frequency components. The power of each frequency component is averaged over all windows, and the frequency with the highest power is identified as the Morse code tone.

3. Butterworth Filter

A Butterworth filter is a type of signal filter that is designed to have a maximally flat frequency response in the passband. This ensures that frequencies within the passband are allowed to pass through without distortion. In this project, a Butterworth filter is used to isolate the Morse code tone from other frequencies. Once the tone frequency is detected, a Butterworth bandpass filter is applied around the detected tone frequency to allow the target frequency to pass through while attenuating other frequencies. The filter's cutoff frequencies are determined based on the tone frequency and bandwidth, and a fifth-order Butterworth filter is used to ensure smooth frequency response with minimal distortion.

4. Hilbert Transform

The Hilbert Transform is a mathematical operation that provides the analytic signal of a real-valued signal. The analytic signal is a complex-valued signal, consisting of the original signal and its Hilbert transform, which shifts the phase by 90 degrees. The magnitude of

this analytic signal represents the envelope of the original signal. In this project, the Hilbert transform is applied to the filtered signal to extract its envelope, which represents the variation in the signal's amplitude over time. This envelope is crucial for detecting the on/off transitions in the signal, which correspond to Morse code elements like dots and dashes. The envelope allows for the identification of binary on/off transitions, which are key to decoding the Morse code symbols.

Now we dive into the methodology for this part of the project.

1. Reading Input Audio Signal

- The audio signal is read from a .wav file using the audioread function. This function retrieves both the audio signal (y) and the sampling frequency (Fs), which defines how many samples are collected per second. In this case, the audio file 'sound4.wav' is used as the input signal.

2. Tone Frequency Detection

- Tone Detection Mode: The setToneFreq variable can be set to 0 to automatically detect the tone frequency used in the Morse code signal. If the frequency is provided, it overrides automatic detection.
- Power Spectral Density (PSD) Calculation: If the tone frequency is not specified, the signal undergoes a tone detection process using the Power Spectral Density (PSD). This technique involves dividing the signal into overlapping windows and applying the Fast Fourier Transform (FFT) to each window to extract the frequency content of the signal.
 - Windowing: A Hanning window is used to minimize edge effects when performing the FFT.
 - Frequency Calculation: The peak of the PSD is used to identify the most dominant frequency, which is assumed to be the Morse code tone.
 - Validation: The detected frequency is validated to ensure it falls within a reasonable range, defined by minToneFreq and maxToneFreq. If the frequency is outside this range or the signal strength is too low, a default frequency (defToneFreq) is used instead.

3. Filtering the Signal

- Bandpass Filtering: A Butterworth bandpass filter is applied to isolate the Morse code tone from the rest of the signal. The filter's cutoff frequencies are computed based on the detected tone frequency (ToneFreq) and a predefined bandwidth (defBandwidth). This helps remove noise and unwanted frequencies outside the Morse tone range.
- Hilbert Transform: The Hilbert transform is applied to the filtered signal to compute its analytic representation. The magnitude of this analytic signal gives the envelope of the

original signal, which is used to detect the presence of Morse code signals (dots and dashes).

4. Binary Signal Conversion

- **Thresholding:** The envelope of the signal is compared to a predefined detection threshold (defDetThr). If the envelope exceeds this threshold, the corresponding part of the signal is classified as active (either a dot or dash); otherwise, it is considered silent.
- **Transition Detection:** The transitions from silent to active states and vice versa are identified to determine the duration of each tone (dot or dash). This is done by finding the indices where the binary signal changes from 0 to 1 (start of a tone) and from 1 to 0 (end of a tone).

5. Classification of Dits and Dahs

- **Duration Thresholding:** The durations of the detected tones are analyzed to classify them as dots (dits) or dashes (dahs). The average duration of dots (avrg_dit_len) is computed, and a threshold (thr) is established to distinguish between the two. Dashes are considered to have a duration longer than dots.
- **Outlier Removal:** Outliers in the duration of dots and dashes are removed using a standard deviation-based approach. Any tone whose duration deviates significantly from the average is discarded.
- **Statistical Measures:** The average, standard deviation, and range of durations for both dots and dashes are computed. These statistical measures help to validate the classification and ensure the tones are correctly identified.

6. Silent Intervals and Word Separation

- **Silence Detection:** The lengths of silent intervals between consecutive tones are measured. If the silence between two tones exceeds a certain threshold, it is interpreted as a space between Morse code characters or words.
- **Word Separation:** If the silence duration is longer than a predefined threshold (e.g., 4 times the average dash length), a word separation (/) is inserted into the decoded message.

7. Morse Code Decoding

- **Character Construction:** Based on the classification of dots and dashes, the Morse code is built character by character. If a tone is classified as a dot, a . is appended to the character. If it is a dash, a - is appended.
- **Character Separation:** When a significant silent interval is detected, the current character is appended to the decoded message, and the process is reset for the next character.
- **Word Separation:** If a long enough silent interval (indicating a word space) is detected, a space () or / is added between words.

8. Output and Display

- **Decoded Message:** After all Morse code symbols are processed, the final decoded message is displayed in the console. The decoded Morse code is stored in the variable `morseCode`, and it is printed for the user to view.

GUI

GUI methodology

At first we designed the App interface in '**App Designer**'. The using callback functions the GUI was developed. Here a step by step summery of that

1. User Inputs a Text Message

- The user types a message into a UI text field (`app.TextMessageEditField`).
- Presses the "**Generate**" button.

2. Text is Converted to Morse Code

The `GenerateButtonPushed` function:

- Converts the input text to lowercase.
- Loops through each character.
- Uses a pre-defined struct `morse` to map characters to Morse code (. and -).
- Constructs a `morseCode` string.
- Displays it in `app.MorseCodeLabel`.

3. Morse Code is Converted to Audio

In the same `GenerateButtonPushed` function:

For each Morse symbol:

- . short beep (dot): 0.1s sine wave.
- - long beep (dash): 0.3s sine wave.
- ' ' (space between letters) :silence of 0.3s.
- '/' (space between words) :silence of 0.7s.

Generates full audio waveform.

Saves it as '`Morse_code_audio.wav`'.

4. User Plays Morse Code Audio

"Play Audio" runs the PlayAudioButtonPushed function:

5. User Uploads Morse Audio for Decoding

Pressing the **"Upload Audio File"** button opens a file picker.

The UploadAudioFileButtonPushed function:

- Reads the audio file using audioread().
- Detects where tones occur using amplitude thresholding.
- Measures durations to distinguish.

Reconstructs the Morse code sequence.

Displays Morse in app.DecodedMessageLabel.

6. Morse Code is Decoded to Text

"Decode" runs the DecodeButtonPushed function:

- Splits the Morse code sequence into characters and words.
- Uses a reverse map (reverseMorse) to find matching letters.
- Constructs the final decoded message.
- Displays the result in app.DecodedTextLabel.

5. Code

Part-1

```
clc

morse=struct;

morse.a='.-';
morse.b='-...';
morse.c='-.-.';
morse.d='-..';
morse.e='.';
morse.f='...-';
morse.g='--.';
morse.h='....';
morse.i='..';
morse.j='---.';
morse.k='-.-';
morse.l='-..';
morse.m='--';
morse.n='-.';
morse.o='---';
morse.p='.-.-';
morse.q='--.-';
morse.r='.-.';
morse.s='...';
morse.t='-';
morse.u='..-';
morse.v='...-';
morse.w='-.-';
morse.x='-.-.';
morse.y='-.-';
morse.z='--..';
morse.nl='----';
```

```

morse.n2='..---';
morse.n3='...--';
morse.n4='....';
morse.n5='.....';
morse.n5='.....';
morse.n6='-....';
morse.n7='--...';
morse.n8='---..';
morse.n9='----.';
morse.n0='-----';
morse.sc='.-.';
morse.scv={'.-.-.',['-.-.-'],['-.-.-'],['-.-.-']}
a=input('press e for Morse encoding and d for morse deconing: ','s')
switch a
    case 'e'
        text=input('enter text: ','s')
        %fprintf('input: %s\n', text)
        text=lower(text);
        l=length(text);
        m=[];

        for i = 1:l
            if text(i)==' '
                m=[m ' '];
            elseif isvarname(text(i))
                m= [m getfield(morse,text(i))];
                m= [m ' '];
            elseif ~isempty(str2num(text(i)))
                m= [m getfield(morse,['n',text(i)])];
                m= [m ' '];
            elseif findstr(text(i),morse.sc)
                m= [m char(morse.scv(findstr(text(i),morse.sc)))];

```

```

        m= [m ' '];
    end
end

fprintf('Output: %s\n', m);

t=1:6000;
dot=sin(t(1:600));
dash=sin(t(1:2000));
ssp= zeros(1,2000);
lsp= zeros(1,4000);
ms=[];
for i= 1:length(m)
    if strcmp(m(i),'.')
        ms=[ms dot ssp];
    elseif strcmp(m(i),'-')
        ms=[ms dash ssp];
    elseif strcmp(m(i),' ')
        ms=[ms lsp];
    elseif strcmp(m(i),'/')
        ms=[ms lsp ssp];
    end
end

end

p=audioplayer(ms,13000);

play(p)

% Save Morse code sound to a .wav file
filename = 'sound4.wav';
audiowrite(filename, ms, 13000);
disp(['Morse code saved as ', filename]);

case 'd'

%decode

code=input('Enter Morse: ','s')

t=1:6000;
dot=sin(t(1:600));

```

```

dash=sin(t(1:2000));

ssp= zeros(1,2000);

lsp= zeros(1,4000);


ms=[];


for i= 1:length(code)

    if strcmp(code(i),'.')
        ms=[ms dot ssp];
    elseif strcmp(code(i),'-')
        ms=[ms dash ssp];
    elseif strcmp(code(i),' ')
        ms=[ms lsp];
    elseif strcmp(code(i),'/')
        ms=[ms lsp ssp];
    end
end

p=audioplayer(ms,13000);

play(p)

dec=[];

code=[code ' '];

lcode=[];


for j= 1:length(code)

    if (strcmp(code(j),' '))|strcmp(code(j),'/')

        for i= double('a'):double('z')

            let=getfield(morse,char(i));

            if strcmp(lcode,let)

                dec=[dec char(i)];

            end

        end

    end

    for i=0:9

```

```

        num=getfield(morse,['n',num2str(i)]);

        if strcmp(lcode,num)

            dec=[dec num2str(i)];

        end

    end

    for i=1:4

        scv=char(morse.scv(i));

        if strcmp(lcode,scv)

            dec=[dec ,morse.sc(i)];

        end

    end

    lcode=[];

else

    lcode=[lcode code(j)];

end

if strcmp(code(j),'/')

    dec=[dec ' '];

end

end

end

fprintf('Decoded Output: %s\n', dec);

end

%sound

```

Part-2

```

% Read the input audio signal

[y, Fs] = audioread('sound4.wav'); % Replace with your actual audio file

% Define tone frequency (use 0 to detect it automatically)

setToneFreq = 0;

% Define Morse timing parameters

dit_len = 0.06; % Length of a dot (seconds)

dah_len = 3 * dit_len; % Length of a dash

spc_len = dit_len; % Space between symbols

```

```

spc_wrd = dah_len; % Space between words

% Tone frequency detection parameters
defToneFreq = 1000; % Default frequency (Hz)
minToneFreq = 500; % Minimum detectable frequency
maxToneFreq = 2500; % Maximum detectable frequency
defBandwidth = 5 / dit_len; % Bandwidth of filter
defDetThr = 0.75; % Detection threshold

% Initialize variables
nsmpl = length(y);
morseCode = cell(1, 1);
MsgToneFreq = zeros(1, 1);
ditstat = zeros(1, 4);
dahstat = zeros(1, 4);
dit_smpl = floor(dit_len * Fs);
curSig = y; % Assign input signal

% Tone detection
if setToneFreq == 0

    % Detect tone using Power Spectral Density (PSD)

    win_size = dit_smpl;
    m = win_size - 1;
    window = 0.5 - 0.5 * cos(2 * pi * (0:m)' / m); % Hanning window
    overlap = ceil(length(window) / 2);
    step = win_size - overlap;
    offset = 1:step:nsmpl - win_size;
    nslc = length(offset);
    P = zeros(win_size, nslc);
    for i = 1:nslc
        P(:, i) = curSig(offset(i):offset(i) + win_size - 1) .* window;
    end

    % Compute Fourier transform and extract positive frequencies
    P = fft(P, [], 1);

```



```

iNyq = floor(win_size / 2);
P = P(1:iNyq, :);
P = mean(conj(P) .* P, 2);
% Find peak frequency
[~, im] = max(P);
ToneFreq = Fs / win_size * (im - 1);
sig_strength = P(im) / median(P);
% Validate detected frequency
if (ToneFreq < minToneFreq) || (ToneFreq > maxToneFreq) || (sig_strength < 2)
    warning('Unreliable tone detection! Check the signal.');
```

ToneFreq = defToneFreq;

```

end
else
    ToneFreq = setToneFreq; % Use predefined frequency
end
MsgToneFreq = ToneFreq;

% Filtering the signal around detected tone frequency
% Compute cutoff frequencies (ensure within valid range)
lowCutoff = max(0, (ToneFreq - defBandwidth / 2) / (Fs / 2));
highCutoff = min(1, (ToneFreq + defBandwidth / 2) / (Fs / 2));
% Validate frequency range
if lowCutoff >= highCutoff
    error('Invalid filter cutoff frequencies. Adjust the bandwidth or tone frequency.');
```

end

```

% Apply 5th order Butterworth bandpass filter
[b, a] = butter(5, [lowCutoff, highCutoff]);
curSig = filtfilt(b, a, curSig);
% Compute the envelope using the Hilbert transform
curSig = hilbert(curSig);
sigEnv = abs(curSig);
% Convert to binary signal based on threshold
```

```

level_threshold = defDetThr * max(sigEnv);
binSig = sigEnv >= level_threshold;

% Detect transitions in the signal
iSta = find(diff(binSig) == 1);
iEnd = find(diff(binSig) == -1);
len_on = iEnd - iSta;

% Estimate duration threshold for dit/dah classification
max_len = max(len_on);
min_len = min(len_on);
thr = (max_len + min_len) / 2;

% Identify dots (dits)
bDit = len_on < thr;
iDit = find(bDit);
avrg_dit_len = mean(len_on(iDit));
std_dit_len = std(len_on(iDit));

% Remove outliers
iout = abs(len_on(iDit) - avrg_dit_len) > 6 * std_dit_len;
bDit(iout) = 0;

% Recalculate
iDit = find(bDit);
avrg_dit_len = mean(len_on(iDit));
avrg_dit = avrg_dit_len / Fs;
std_dit_len = std(len_on(iDit));
ditstat = [avrg_dit, std_dit_len / Fs, min(len_on(iDit)) / Fs, max(len_on(iDit)) / Fs];

% Identify dashes (dahs)
bDah = len_on > thr;
iDah = find(bDah);
avrg_dah_len = mean(len_on(iDah));
std_dah_len = std(len_on(iDah));

% Remove outliers
iout = abs(len_on(iDah) - avrg_dah_len) > 6 * std_dah_len;

```

```

bDah(iout) = 0;

% Recalculate

iDah = find(bDah);

avrg_dah_len = mean(len_on(iDah));

avrg_dah = avrg_dah_len / Fs;

std_dah_len = std(len_on(iDah));

dahstat = [avrg_dah, std_dah_len / Fs, min(len_on(iDah)) / Fs, max(len_on(iDah)) / Fs];

% Detect silence intervals

len_off = iSta(2:end) - iEnd(1:end-1);

len_off(end+1) = length(binSig) - iEnd(end);

% Decode Morse Code

morseC = "";

curMsg = "";

for idx = 1:length(len_on)

    if bDah(idx)

        morseC = strcat(morseC, "-");

    elseif bDit(idx)

        morseC = strcat(morseC, ".");

    end

    % Append the Morse character

    if idx == length(len_on) || len_off(idx) > 0

        curMsg = strcat(curMsg, morseC);

        morseC = ""; % Reset for next character

    end

    % Insert space if a silent interval is detected

    if len_off(idx) > (10 * avrg_dit_len) % Adjust this threshold as needed

        curMsg = strcat(curMsg, " ");

    end

    % Word separation

    if len_off(idx) > 4 * avrg_dah_len

        if idx < length(len_on)

            curMsg = strcat(curMsg, ' '); % ' ' represents word separator

```

```

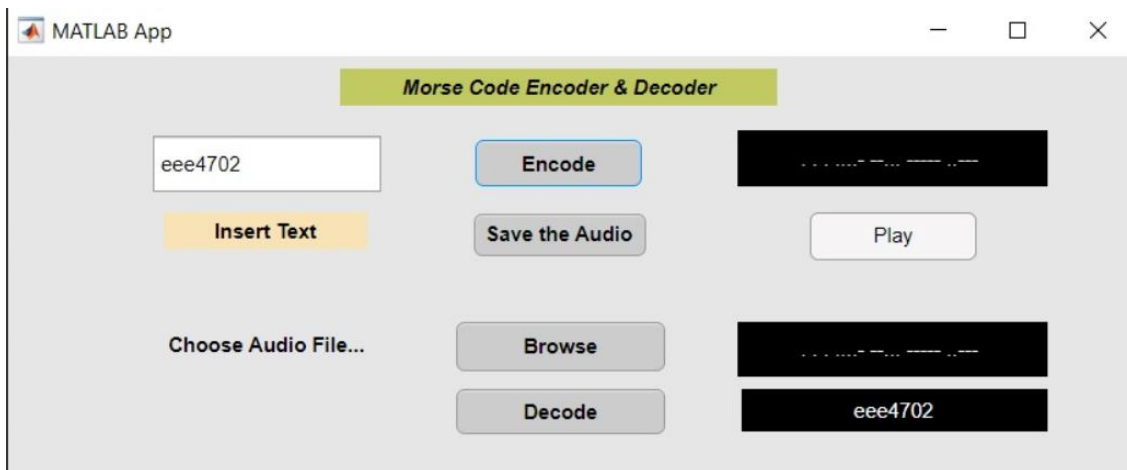
        end
    end
end
morseCode{1} = curMsg;
% Convert Morse code to string and display result
morseCode = char(morseCode{:});
disp(['Decoded Morse Code: ', morseCode]);

```

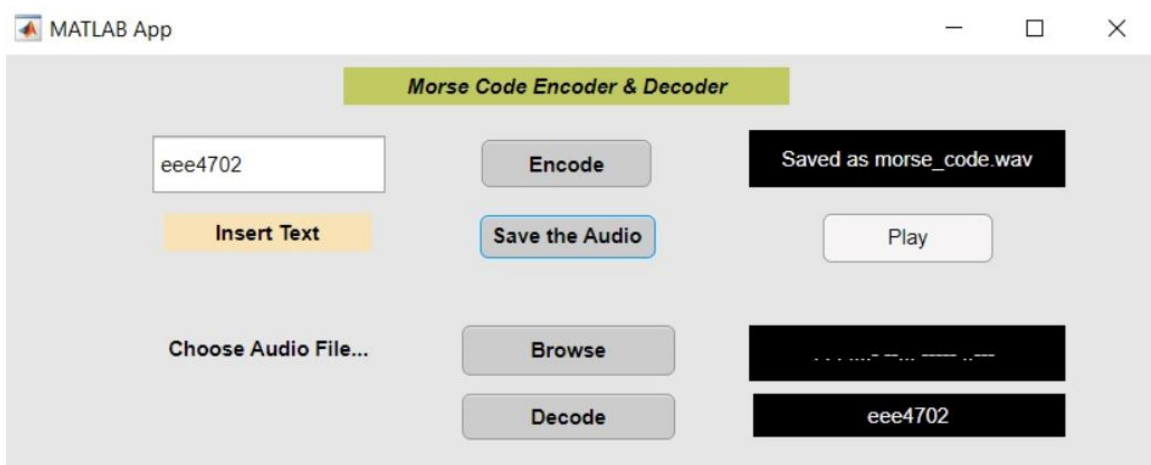
6. Result Analysis

MATLAB GUI:

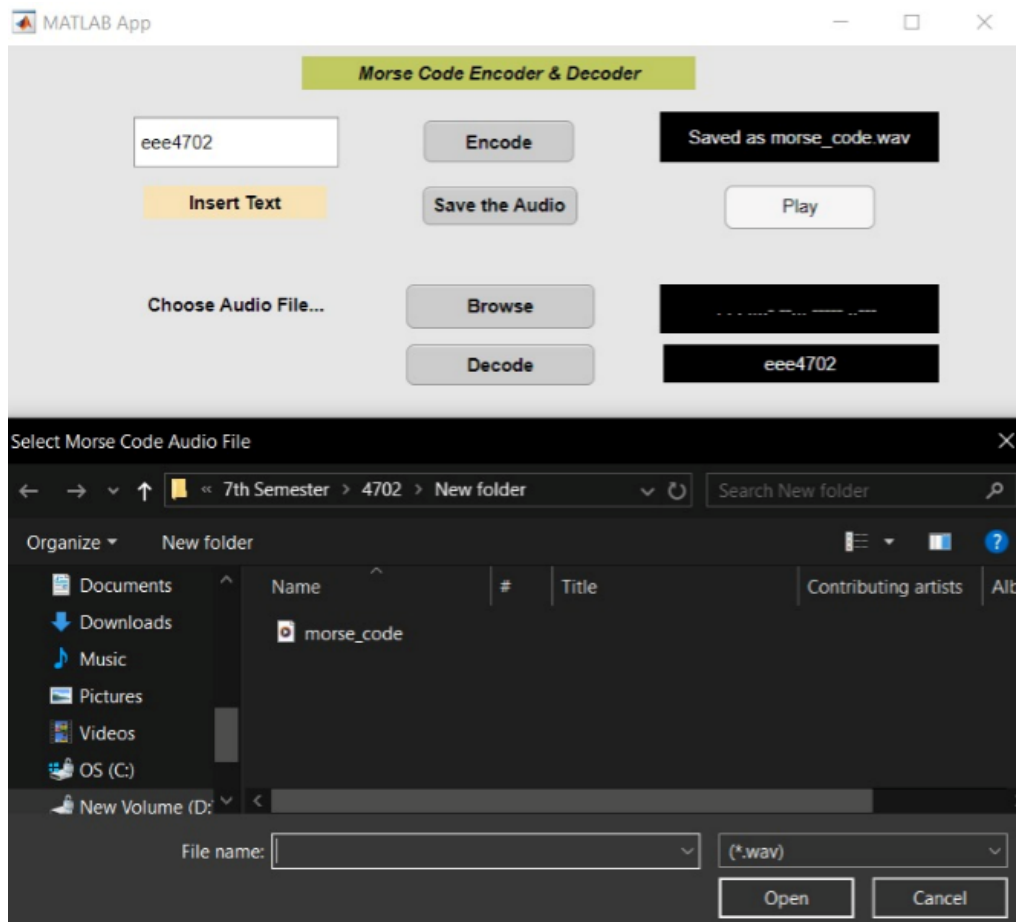
Encoding:



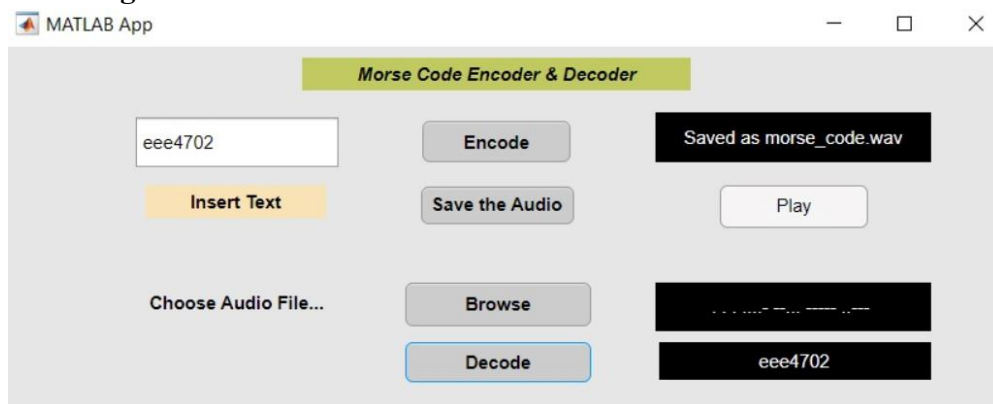
Saving The Audio:



Browsing The Audio File:



Decoding:



7. Conclusion

In conclusion, the signal processing techniques employed in this project, including Power Spectral Density (PSD), Fast Fourier Transform (FFT), Butterworth filtering, and the Hilbert Transform, have successfully been utilized to detect and decode Morse code signals from an audio source. However, one limitation of this approach lies in its sensitivity to noise and interference present in non-ideal conditions, such as background sounds or signal distortions. These environmental factors can significantly affect the accuracy of the frequency detection and filtering process, leading to potential misclassification or failure in decoding the Morse code correctly. Furthermore, the algorithm depends on fixed parameters, such as the predefined tone frequency range, window size for the FFT, and the bandwidth for the Butterworth filter. In cases where the signal characteristics vary, such as in a dynamic environment with fluctuating signal conditions, these fixed parameters may not provide optimal performance.

To address these limitations, future improvements could focus on implementing adaptive filtering techniques that dynamically adjust to the frequency content of the signal. Additionally, employing machine learning models to learn and adapt the system parameters in real-time could enhance the robustness of the system, ensuring accurate detection even in the presence of noise or when signal characteristics change. Moreover, optimizing the detection algorithm with more sophisticated noise reduction methods or multi-band filtering could further increase the overall reliability and efficiency of Morse code decoding in real-world applications.

8. Team Contribution

1. Sayeed Hossain Sayor:
ID: 200021236
Contribution:
Matlab Coding for Morse code Generator.
Morse Audio Generation.
Preparing Presentation Slides.
2. M.M. Sadman Shafi:
ID: 200021240
Contribution:
Morse Audio Processing.
Matlab Coding for Morse Code Decoder.
Project Report Writing.
3. MD Wahiduzzaman :
ID: 200021228
Contribution:
Matlab Coding for Graphical User Interface.
Preparing Presentation Slides.
Project Report Writing.
4. Shafayet Sadik Sowad:
ID: 200021248
Contribution:
Matlab Coding for Graphical User Interface.
Preparing Presentation Slides.
Project Report Writing.
5. Mohammad Saed Jama
ID: 200021260
Contribution:
Presentation Slides

9. Referencing and Citations

- a. <https://www.mathworks.com/matlabcentral/fileexchange/111870-morse-decoder>.

